

Sprint 20 Structured Notes: Classes, `this`, Inheritance, `super`, Static Members, and Prototype Awareness

0. What Sprint 20 Is About

Sprint 20 is about learning how to create **reusable object blueprints** in JavaScript.

Before this sprint, you already learned how to create objects manually:

```
const student = {  
  name: "Sangeeth",  
  score: 90  
};
```

That works for one object. But if you need many similar objects, writing each one manually becomes repetitive.

Sprint 20 teaches you how to use **classes** so you can create many objects from one shared plan.

Main goal:

Create reusable blueprints for objects.

By the end of Sprint 20, you should understand:

- classes
- class declarations
- class expressions
- PascalCase class names
- constructors
- the `new` keyword
- instances
- properties
- methods
- `this`
- inheritance
- `extends`
- `super()`
- method overriding
- calling parent methods with `super.methodName()`

- static methods
- static properties
- instance members vs static members
- basic prototype awareness
- common beginner class mistakes

1. Current Progress Before Sprint 20

Sprint 20 depends on the skills from the earlier sprints.

Earlier Sprint Area	What You Learned	How It Helps in Sprint 20
Sprints 1–4	Values, variables, strings, numbers, booleans, conditions	Class objects store values and make decisions.
Sprint 5	Functions, parameters, return values, scope, closures	Class methods are functions inside a class.
Sprints 6–7	Arrays and loops	You can create arrays of class instances and loop through them.
Sprint 8	Objects, properties, methods, <code>this</code> basics	Classes create objects with properties and methods.
Sprint 9	Callbacks and higher-order methods	Class methods can be used with array methods and callbacks.
Sprint 10	Sets and Maps	Class instances can be stored in Sets and Maps.
Sprints 11–17	DOM, events, forms, storage, CRUD	Classes can organize app logic for browser projects.
Sprint 18	Debugging	Class errors often involve <code>this</code> , <code>new</code> , or <code>super()</code> .
Sprint 19	Regex	Static helper methods can validate text patterns.

Simple summary:

Before Sprint 20, you learned how to make objects manually. Sprint 20 teaches you how to design reusable object blueprints.

Part A: The Core Idea of Classes

2. What Is a Class?

A **class** is a blueprint for creating objects.

Simple meaning:

A class is a plan. Objects are created from that plan.

Example:

```
class Dog {  
  // blueprint  
}
```

This class is not one dog. It is a plan for making dog objects.

A real example:

```
class Dog {  
  constructor(name) {  
    this.name = name;  
  }  
  
  bark() {  
    console.log(`${this.name} says woof!`);  
  }  
}  
  
const dog1 = new Dog("Gino");  
const dog2 = new Dog("Luna");  
  
dog1.bark();  
dog2.bark();
```

Expected output:

```
Gino says woof!  
Luna says woof!
```

Simple meaning:

```
new Dog("Gino")
```

creates a new object using the `Dog` class blueprint.

3. Why Classes Exist

Without classes, you may create objects manually like this:

```
const dog1 = {  
  name: "Gino",  
  bark() {  
    console.log(`${this.name} says woof!`);  
  }  
};  
  
const dog2 = {  
  name: "Luna",  
  bark() {  
    console.log(`${this.name} says woof!`);  
  }  
};
```

This works, but the same structure is repeated.

Both objects have:

- a `name` property
- a `bark()` method

A class removes the repetition:

```
class Dog {  
  constructor(name) {  
    this.name = name;  
  }  
  
  bark() {  
    console.log(`${this.name} says woof!`);  
  }  
}
```

```
const dog1 = new Dog("Gino");
const dog2 = new Dog("Luna");
```

Now one class can create many dog objects.

Beginner rule:

Use a class when many objects share the same shape and behavior.

4. Object Shape and Behavior

When learning classes, separate two ideas:

Idea	Meaning	Example
Shape	What data the object has	<code>name</code> , <code>score</code> , <code>done</code>
Behavior	What the object can do	<code>bark()</code> , <code>passed()</code> , <code>complete()</code>

Example:

```
class Student {
  constructor(name, score) {
    this.name = name;
    this.score = score;
  }

  passed() {
    return this.score >= 70;
  }
}
```

The shape is:

```
this.name
this.score
```

The behavior is:

```
passed()
```

Simple meaning:

A class describes what an object has and what an object can do.

Part B: Class Syntax

5. Class Declaration

A **class declaration** creates a named class.

Syntax:

```
class ClassName {  
    // constructor and methods go here  
}
```

Example:

```
class User {  
    constructor(username) {  
        this.username = username;  
    }  
}
```

This creates a class named `User`.

Beginner rule:

Most of the time, use class declarations.

6. Class Expression

A **class expression** stores a class in a variable.

Example:

```
const User = class {  
    constructor(username) {  
        this.username = username;  
    }  
}
```

```
}  
};
```

This also creates a class, but the class is assigned to a variable.

You can also use a named class expression:

```
const User = class UserClass {  
  constructor(username) {  
    this.username = username;  
  }  
};
```

For beginners, this is less common.

Beginner rule:

Prefer `class User {}` unless you have a clear reason to use a class expression.

7. Class Name Convention: PascalCase

Class names usually use **PascalCase**.

Good class names:

```
Student  
Dog  
Task  
ShoppingCart  
OnlineStudent
```

Weak class names:

```
student  
dog  
task  
shoppingCart
```

Why PascalCase?

Because it visually tells other developers:

This is a class or constructor-style blueprint.

Example:

```
class ShoppingCart {  
  constructor() {  
    this.items = [];  
  }  
}
```

Part C: constructor

8. What Is a Constructor?

The `constructor` is a special method inside a class.

It runs automatically when you create a new object with `new`.

Example:

```
class Student {  
  constructor(name, score) {  
    this.name = name;  
    this.score = score;  
  }  
}  
  
const student = new Student("Sangeeth", 90);
```

This line:

```
new Student("Sangeeth", 90)
```

runs the constructor:

```
constructor(name, score) {  
  this.name = name;  
  this.score = score;  
}
```


Simple meaning:

The constructor sets up the new object.

9. Constructor Parameters and Arguments

A constructor can receive parameters just like a normal function.

Example:

```
class Student {  
  constructor(name, score) {  
    this.name = name;  
    this.score = score;  
  }  
}  
  
const student = new Student("Sangeeth", 90);
```

Here:

Code	Meaning
name	Parameter
score	Parameter
"Sangeeth"	Argument
90	Argument

The values flow like this:

```
"Sangeeth" -> name -> this.name  
90 -> score -> this.score
```

After object creation, the object has:

```
student.name; // "Sangeeth"  
student.score; // 90
```

10. Constructor Dry Run

Code:

```
class Student {  
  constructor(name, score) {  
    this.name = name;  
    this.score = score;  
  }  
}  
  
const student = new Student("Sangeeth", 90);
```

Step-by-step:

Step	What Happens
1	JavaScript sees <code>new Student(...)</code> .
2	JavaScript creates a new empty object.
3	JavaScript runs the <code>constructor</code> .
4	<code>name</code> receives <code>"Sangeeth"</code> .
5	<code>score</code> receives <code>90</code> .
6	<code>this.name = name</code> stores the name on the object.
7	<code>this.score = score</code> stores the score on the object.
8	The final object is stored in <code>student</code> .

Conceptual result:

```
{  
  name: "Sangeeth",  
  score: 90  
}
```

11. Default Values in Constructors

Constructors can use default parameter values.

Example:

```
class User {  
  constructor(username = "Guest") {  
    this.username = username;  
  }  
}  
  
const user1 = new User("Sangeeth");  
const user2 = new User();  
  
console.log(user1.username);  
console.log(user2.username);
```

Expected output:

```
Sangeeth  
Guest
```

Simple meaning:

A default value is used when no argument is passed.

Part D: The `new` Keyword

12. What Does `new` Do?

The `new` keyword creates an instance from a class.

Example:

```
const student = new Student("Sangeeth", 90);
```

Simple meaning:

`new` tells JavaScript to create a new object from the class blueprint.

The `new` keyword does these things:

1. Creates a new empty object.
2. Connects the object to the class prototype.
3. Runs the constructor.
4. Makes `this` refer to the new object.

5. Returns the new object.

Beginner version:

`new` builds the object and runs the setup code.

13. Instance

An **instance** is an object created from a class.

Example:

```
class Dog {  
  constructor(name) {  
    this.name = name;  
  }  
}  
  
const dog = new Dog("Gino");
```

Here:

dog

is an instance of the `Dog` class.

You can create many instances:

```
const dog1 = new Dog("Gino");  
const dog2 = new Dog("Luna");  
const dog3 = new Dog("Max");
```

Each object is separate:

```
console.log(dog1.name); // Gino  
console.log(dog2.name); // Luna  
console.log(dog3.name); // Max
```

Simple meaning:

A class is the blueprint. An instance is one object made from that blueprint.

14. Forgetting `new`

A common beginner mistake is forgetting `new`.

Wrong:

```
const dog = Dog("Gino");
```

Correct:

```
const dog = new Dog("Gino");
```

With classes, JavaScript expects you to use `new`.

Beginner rule:

When creating an object from a class, use `new`.

Part E: `this`

15. What Does `this` Mean?

Inside a class method, `this` usually means:

the current object using the method.

Example:

```
class Student {  
  constructor(name, score) {  
    this.name = name;  
    this.score = score;  
  }  
  
  introduce() {  
    return `Hi, I am ${this.name}.`;  
  }  
}
```

```
const student = new Student("Sangeeth", 90);

console.log(student.introduce());
```

Expected output:

Hi, I am Sangeeth.

Inside this method:

```
introduce() {
  return `Hi, I am ${this.name}.`;
}
```

`this.name` means:

Get the `name` property from this specific student object.

16. Same Method, Different Objects

The same method can work with different objects.

Example:

```
class Student {
  constructor(name) {
    this.name = name;
  }

  introduce() {
    return `Hi, I am ${this.name}.`;
  }
}

const student1 = new Student("Sangeeth");
const student2 = new Student("Ada");

console.log(student1.introduce());
console.log(student2.introduce());
```

Expected output:

```
Hi, I am Sangeeth.  
Hi, I am Ada.
```

Why?

Because when this runs:

```
student1.introduce()
```

this is student1.

When this runs:

```
student2.introduce()
```

this is student2.

Simple meaning:

this depends on which object is calling the method.

17. Why this Is Needed

Without this, the value is not stored on the object.

Wrong:

```
class Student {  
  constructor(name) {  
    name = name;  
  }  
}  
  
const student = new Student("Sangeeth");  
console.log(student.name);
```

Expected output:

```
undefined
```

Why?

Because this line does not store anything on the object:

```
name = name;
```

Correct:

```
class Student {  
  constructor(name) {  
    this.name = name;  
  }  
}  
  
const student = new Student("Sangeeth");  
console.log(student.name);
```

Expected output:

```
Sangeeth
```

Beginner rule:

Use `this.propertyName` when storing data inside the object.

Part F: Properties

18. What Is a Property?

A property is data stored inside an object.

Example:

```
class Student {  
  constructor(name, score) {  
    this.name = name;  
    this.score = score;  
  }  
}
```



```
}  
}
```

The properties are:

```
this.name  
this.score
```

After creating an instance:

```
const student = new Student("Sangeeth", 90);
```

You can read the properties:

```
console.log(student.name);  
console.log(student.score);
```

Expected output:

```
Sangeeth  
90
```

Simple meaning:

Properties are the data each object stores.

19. Updating Instance Properties

You can update object properties after creation.

Example:

```
class Student {  
  constructor(name, score) {  
    this.name = name;  
    this.score = score;  
  }  
}  
  
const student = new Student("Sangeeth", 90);
```

```
student.score = 95;  
  
console.log(student.score);
```

Expected output:

```
95
```

Simple meaning:

Class instances are objects, so their properties can be read and changed like normal object properties.

Part G: Methods

20. What Is a Method?

A method is a function inside a class.

Example:

```
class Student {  
  constructor(name, score) {  
    this.name = name;  
    this.score = score;  
  }  
  
  passed() {  
    return this.score >= 70;  
  }  
}
```

Here:

```
passed()
```

is a method.

You call it like this:

```
const student = new Student("Sangeeth", 90);
console.log(student.passed());
```

Expected output:

```
true
```

Simple meaning:

A method is behavior that belongs to an object.

21. Methods Can Return Values

Methods can return values just like normal functions.

Example:

```
class Student {
  constructor(name, score) {
    this.name = name;
    this.score = score;
  }

  describe() {
    return `${this.name} has a score of ${this.score}.`;
  }
}

const student = new Student("Sangeeth", 90);
console.log(student.describe());
```

Expected output:

```
Sangeeth has a score of 90.
```

Beginner rule:

Use `return` when a method should give a value back.

22. Methods Can Change Object State

A method can also update the object.

Example:

```
class Task {  
  constructor(text) {  
    this.text = text;  
    this.done = false;  
  }  
  
  complete() {  
    this.done = true;  
  }  
}  
  
const task = new Task("Study classes");  
console.log(task.done);  
  
task.complete();  
console.log(task.done);
```

Expected output:

```
false  
true
```

This method changes the object:

```
complete() {  
  this.done = true;  
}
```

Simple meaning:

Methods can read object data or change object data.

23. `return` vs `console.log()` in Methods

This method returns a reusable value:

```
class Student {
  constructor(score) {
    this.score = score;
  }

  passed() {
    return this.score >= 70;
  }
}
```

This method only prints:

```
class Student {
  constructor(score) {
    this.score = score;
  }

  printStatus() {
    console.log(this.score >= 70);
  }
}
```

Comparison:

Tool	Meaning
<code>return</code>	Gives a value back so other code can use it.
<code>console.log()</code>	Prints something for inspection.

Beginner rule:

Use `return` for logic. Use `console.log()` for checking output while learning.

Part H: Inheritance

24. What Is Inheritance?

Inheritance means one class can reuse behavior from another class.

Simple meaning:

A child class can receive properties and methods from a parent class.

Example:

```
class Animal {
  constructor(name) {
    this.name = name;
  }

  eat() {
    return `${this.name} is eating.`;
  }
}

class Dog extends Animal {
  bark() {
    return `${this.name} says woof!`;
  }
}

const dog = new Dog("Gino");

console.log(dog.eat());
console.log(dog.bark());
```

Expected output:

```
Gino is eating.
Gino says woof!
```

Dog gets eat() from Animal.

Dog adds bark() itself.

25. Parent Class and Child Class

In inheritance, there is usually a parent class and a child class.

Example:

```
class Animal {
  // parent class
}
```

```
class Dog extends Animal {  
  // child class  
}
```

Term	Meaning	Example
Parent class	The general class	Animal
Child class	The more specific class	Dog
Base class	Another name for parent class	Animal
Subclass	Another name for child class	Dog

Simple meaning:

Dog is a specific kind of Animal.

26. extends

The `extends` keyword creates inheritance.

Example:

```
class Dog extends Animal {  
  bark() {  
    return `${this.name} says woof!`;  
  }  
}
```

This means:

Dog inherits from Animal.

So this works:

```
const dog = new Dog("Gino");  
dog.eat();
```

Even if `eat()` was written in `Animal`, the `Dog` instance can use it.

Beginner rule:

Use `extends` when one class is a more specific version of another class.

27. When Inheritance Makes Sense

Inheritance makes sense when there is an **is-a** relationship.

Good examples:

```
Dog is an Animal.  
OnlineStudent is a Student.  
AdminUser is a User.  
SavingsAccount is an Account.
```

Weak examples:

```
Car is an Engine.           // Not correct.  
Task is a Calendar.        // Not correct.  
Student is a Course.       // Not correct.
```

Beginner rule:

Use inheritance only when the child really is a type of the parent.

Part I: `super`

28. What Is `super()`?

`super()` calls the parent class constructor.

Example:

```
class Animal {  
    constructor(name) {  
        this.name = name;  
    }  
}  
  
class Dog extends Animal {  
    constructor(name, breed) {  
        super(name);  
    }  
}
```



```
        this.breed = breed;
    }
}

const dog = new Dog("Gino", "Labrador");

console.log(dog.name);
console.log(dog.breed);
```

Expected output:

```
Gino
Labrador
```

This line:

```
super(name);
```

calls the parent constructor:

```
constructor(name) {
    this.name = name;
}
```

Then the child constructor adds:

```
this.breed = breed;
```

Simple meaning:

`super()` lets the parent class set up its part of the object.

29. `super()` Must Come Before `this`

In a child class constructor, you must call `super()` before using `this`.

Wrong:

```
class Dog extends Animal {  
  constructor(name, breed) {  
    this.breed = breed;  
    super(name);  
  }  
}
```

Correct:

```
class Dog extends Animal {  
  constructor(name, breed) {  
    super(name);  
    this.breed = breed;  
  }  
}
```

Beginner rule:

In a child constructor, call `super()` first.

30. Child Class Without Its Own Constructor

A child class does not always need its own constructor.

Example:

```
class Animal {  
  constructor(name) {  
    this.name = name;  
  }  
  
  eat() {  
    return `${this.name} is eating.`;  
  }  
}  
  
class Dog extends Animal {  
  bark() {  
    return `${this.name} says woof!`;  
  }  
}
```

```
const dog = new Dog("Gino");
console.log(dog.eat());
console.log(dog.bark());
```

Expected output:

```
Gino is eating.
Gino says woof!
```

Since `Dog` does not define its own constructor, JavaScript uses the parent constructor automatically.

Part J: Method Overriding

31. What Is Method Overriding?

Method overriding means a child class replaces a method from the parent class.

Example:

```
class Animal {
  speak() {
    return "The animal makes a sound.";
  }
}

class Dog extends Animal {
  speak() {
    return "The dog barks.";
  }
}

const dog = new Dog();
console.log(dog.speak());
```

Expected output:

```
The dog barks.
```

The child version wins because `Dog` has its own `speak()` method.

Simple meaning:

A child class can customize behavior inherited from a parent class.

32. Calling a Parent Method with `super.methodName()`

A child method can call a parent method.

Example:

```
class Animal {  
  speak() {  
    return "The animal makes a sound.";  
  }  
}  
  
class Dog extends Animal {  
  speak() {  
    return `${super.speak()} The dog barks.`;  
  }  
}  
  
const dog = new Dog();  
console.log(dog.speak());
```

Expected output:

```
The animal makes a sound. The dog barks.
```

This part:

```
super.speak()
```

calls the parent method.

Simple meaning:

`super.methodName()` lets a child method reuse the parent method.

Part K: Static Methods

33. What Is a Static Method?

A static method belongs to the class itself, not to one object instance.

Example:

```
class MathHelper {  
  static double(number) {  
    return number * 2;  
  }  
}  
  
console.log(MathHelper.double(5));
```

Expected output:

10

You call a static method on the class:

```
MathHelper.double(5)
```

Not on an instance.

Wrong:

```
const helper = new MathHelper();  
helper.double(5);
```

That fails because `double()` belongs to the class, not the instance.

34. When to Use Static Methods

Use a static method when the behavior belongs to the class idea, but does not need data from one specific object.

Example:

```
class Student {
  static isPassingScore(score) {
    return score >= 70;
  }
}

console.log(Student.isPassingScore(90));
console.log(Student.isPassingScore(50));
```

Expected output:

```
true
false
```

This method does not need one specific student object.

It only checks a number.

Beginner rule:

Use `static` when the method does not need `this.name`, `this.score`, or other instance data.

35. Instance Method vs Static Method

Instance method:

```
class Student {
  constructor(name, score) {
    this.name = name;
    this.score = score;
  }

  passed() {
    return this.score >= 70;
  }
}

const student = new Student("Sangeeth", 90);
console.log(student.passed());
```

Static method:

```
class Student {
  static isPassingScore(score) {
    return score >= 70;
  }
}

console.log(Student.isPassingScore(90));
```

Comparison:

Method Type	Called On	Uses Instance Data?	Example
Instance method	Object instance	Usually yes	<code>student.passed()</code>
Static method	Class itself	Usually no	<code>Student.isPassingScore(90)</code>

Rule:

Needs data from one object? Use an instance method.
Does not need one object? Use a static method.

Part L: Static Properties

36. What Is a Static Property?

A static property is data stored on the class itself, not on each object.

Example:

```
class Student {
  static passingScore = 70;

  constructor(name, score) {
    this.name = name;
    this.score = score;
  }

  passed() {
    return this.score >= Student.passingScore;
  }
}
```

```
const student = new Student("Sangeeth", 90);
console.log(student.passed());
console.log(Student.passingScore);
```

Expected output:

```
true
70
```

This property belongs to the class:

```
Student.passingScore
```

Not to each individual student.

Simple meaning:

A static property stores class-level information.

37. Static Property vs Instance Property

Example:

```
class Student {
  static schoolName = "JavaScript Academy";

  constructor(name) {
    this.name = name;
  }
}

const student1 = new Student("Sangeeth");
const student2 = new Student("Ada");

console.log(student1.name);
console.log(student2.name);
console.log(Student.schoolName);
```

Expected output:


```
Sangeeth  
Ada  
JavaScript Academy
```

Here:

```
this.name
```

is different for each object.

But:

```
Student.schoolName
```

belongs to the class.

Comparison:

Property Type	Stored On	Example
Instance property	Each object	<code>student.name</code>
Static property	Class itself	<code>Student.schoolName</code>

Part M: Prototype Awareness

38. Why Prototypes Matter

JavaScript classes are built on top of JavaScript's prototype system.

As a beginner, you do not need deep prototype mastery yet.

But you should know this:

Class methods are shared through the class prototype instead of being copied into every object.

Example:

```

class Dog {
  constructor(name) {
    this.name = name;
  }

  bark() {
    console.log(`${this.name} says woof!`);
  }
}

const dog1 = new Dog("Gino");
const dog2 = new Dog("Luna");

```

Both objects can use:

```
bark()
```

But JavaScript does not need to copy a separate `bark()` function into each object.

The method is shared through the prototype.

Simple meaning:

Prototypes let objects share behavior efficiently.

39. Older Prototype Style

Before modern class syntax, JavaScript often used constructor functions and prototypes.

Older style:

```

function Dog(name) {
  this.name = name;
}

Dog.prototype.bark = function () {
  console.log(`${this.name} says woof!`);
};

const dog = new Dog("Gino");
dog.bark();

```

Modern class style:

```
class Dog {  
  constructor(name) {  
    this.name = name;  
  }  
  
  bark() {  
    console.log(`${this.name} says woof!`);  
  }  
}  
  
const dog = new Dog("Gino");  
dog.bark();
```

Both are connected to prototypes.

Beginner rule:

Use modern `class` syntax. Know that prototypes are underneath.

40. Avoid Complex Prototype Manipulation for Now

You may see advanced code like this:

```
Dog.prototype.someMethod = function () {  
  // advanced pattern  
};
```

Or:

```
Object.setPrototypeOf(obj, otherObj);
```

As a beginner, do not focus on changing prototypes directly.

Why?

Because it can make code harder to understand and debug.

Beginner rule:

Learn classes first. Treat prototypes as background knowledge for now.

Part N: Full Beginner Example — Task

41. Task Class Example

This example connects Sprint 20 to CRUD and browser app thinking.

```
class Task {
  constructor(text) {
    this.text = text;
    this.done = false;
    this.createdAt = new Date().toISOString();
  }

  complete() {
    this.done = true;
  }

  describe() {
    const status = this.done ? "done" : "not done";
    return `${this.text} is ${status}.`;
  }

  static fromText(text) {
    return new Task(text.trim());
  }
}

const task = Task.fromText("  Study Sprint 20  ");

console.log(task.describe());

task.complete();

console.log(task.describe());
console.log(task.createdAt);
```

Expected output pattern:

```
Study Sprint 20 is not done.
Study Sprint 20 is done.
2026-...
```

42. Task Example Breakdown

Class name:

```
class Task
```

This creates a blueprint for task objects.

Constructor:

```
constructor(text) {  
  this.text = text;  
  this.done = false;  
  this.createdAt = new Date().toISOString();  
}
```

This sets up each new task.

Instance properties:

```
this.text  
this.done  
this.createdAt
```

These belong to each individual task.

Instance method:

```
complete() {  
  this.done = true;  
}
```

This changes one task object.

Instance method with return value:

```
describe() {  
  const status = this.done ? "done" : "not done";  
  return `${this.text} is ${status}.`;  
}
```

This returns a readable description.

Static method:

```
static fromText(text) {  
  return new Task(text.trim());  
}
```

This creates a task from cleaned text.

Simple meaning:

`Task.fromText()` is a class-level helper for creating a clean task object.

Part O: Full Inheritance Example — Student and OnlineStudent

43. Parent Class: Student

```
class Student {  
  static passingScore = 70;  
  
  constructor(name, score) {  
    this.name = name;  
    this.score = score;  
  }  
  
  passed() {  
    return this.score >= Student.passingScore;  
  }  
  
  describe() {  
    return `${this.name} has a score of ${this.score}.`;  
  }  
  
  static isValidScore(score) {  
    return score >= 0 && score <= 100;  
  }  
}
```

This class has:

- a static property: `passingScore`
 - a constructor: `constructor(name, score)`
 - instance properties: `this.name`, `this.score`
 - instance methods: `passed()`, `describe()`
 - a static method: `isValidScore(score)`
-

44. Child Class: `OnlineStudent`

```
class OnlineStudent extends Student {  
  constructor(name, score, platform) {  
    super(name, score);  
    this.platform = platform;  
  }  
  
  describe() {  
    return `${super.describe()} Platform: ${this.platform}.`;  
  }  
}
```

This class:

- extends `Student`
 - calls the parent constructor with `super(name, score)`
 - adds a new property: `platform`
 - overrides `describe()`
 - uses `super.describe()` to call the parent version
-

45. Using the Student Classes

```
const student = new Student("Sangeeth", 90);  
const onlineStudent = new OnlineStudent("Ada", 95, "Zoom");  
  
console.log(student.describe());  
console.log(student.passed());  
  
console.log(onlineStudent.describe());  
console.log(onlineStudent.passed());
```

```
console.log(Student.isValidScore(105));  
console.log(Student.isValidScore(85));
```

Expected output:

```
Sangeeth has a score of 90.  
true  
Ada has a score of 95. Platform: Zoom.  
true  
false  
true
```

Part P: Common Beginner Mistakes

46. Mistake 1: Forgetting `new`

Wrong:

```
const dog = Dog("Gino");
```

Correct:

```
const dog = new Dog("Gino");
```

Explanation:

Classes need `new` when creating an instance.

47. Mistake 2: Forgetting `this`

Wrong:

```
class Student {  
  constructor(name) {  
    name = name;  
  }  
}
```


Correct:

```
class Student {  
    constructor(name) {  
        this.name = name;  
    }  
}
```

Explanation:

`this.name` stores the value on the object.

48. Mistake 3: Calling a Static Method on an Instance

Wrong:

```
class Student {  
    static isPassing(score) {  
        return score >= 70;  
    }  
}  
  
const student = new Student();  
student.isPassing(90);
```

Correct:

```
Student.isPassing(90);
```

Explanation:

Static methods belong to the class, not the instance.

49. Mistake 4: Forgetting `super()` in a Child Constructor

Wrong:

```
class Dog extends Animal {  
    constructor(name) {
```

```
    this.name = name;
  }
}
```

Correct:

```
class Dog extends Animal {
  constructor(name) {
    super(name);
  }
}
```

Explanation:

In a child constructor, call `super()` before using `this`.

50. Mistake 5: Using Inheritance When It Does Not Fit

Weak design:

```
class Engine extends Car {
  // weak relationship
}
```

Better thinking:

```
A car has an engine.
An engine is not a car.
```

Inheritance should mean:

```
Child is a type of parent.
```

Good design:

```
class Dog extends Animal {
  // Dog is an Animal
}
```

Beginner rule:

Use inheritance for “is-a” relationships, not “has-a” relationships.

51. Mistake 6: Confusing Static Properties and Instance Properties

Wrong expectation:

```
class Student {  
  static schoolName = "JavaScript Academy";  
}  
  
const student = new Student();  
console.log(student.schoolName);
```

This does not work as expected because `schoolName` is on the class.

Correct:

```
console.log(Student.schoolName);
```

Explanation:

Static properties are read from the class itself.

Part Q: Debugging Class Code

52. Common Class Error Types

Class code often creates these errors:

Error Type	Common Cause
ReferenceError	Class name or variable name is misspelled.
TypeError	Method is called on the wrong value.
TypeError	Static method is called on an instance.
ReferenceError	<code>this</code> is used before <code>super()</code> in a child constructor.

53. Debugging Checklist

When class code breaks, ask:

1. Did I use `new`?
2. Did I spell the class name correctly?
3. Did I store data with `this.propertyName`?
4. Am I calling an instance method on an instance?
5. Am I calling a static method on the class?
6. If using `extends`, did I call `super()` first?
7. Is inheritance actually the right design?
8. Did I return the value I expected from the method?

Example checks:

```
console.log(student);  
console.log(student instanceof Student);  
console.log(typeof student.describe);
```

Part R: Minimum Passing Standard

54. Code You Should Be Able to Explain

You understand Sprint 20 when you can explain this code line by line:

```
class Dog {  
  constructor(name) {  
    this.name = name;  
  }  
  
  bark() {  
    console.log(`${this.name} says woof!`);  
  }  
}  
  
const dog = new Dog("Gino");  
dog.bark();  
  
class ServiceDog extends Dog {  
  constructor(name, job) {  
    super(name);  
    this.job = job;  
  }  
}
```

```

    }

    work() {
      console.log(`${this.name} works as a ${this.job}.`);
    }
  }
}

const serviceDog = new ServiceDog("Luna", "guide dog");
serviceDog.bark();
serviceDog.work();

```

Expected output:

```

Gino says woof!
Luna says woof!
Luna works as a guide dog.

```

You should be able to explain:

- what `class Dog` means
- what the constructor does
- why `this.name = name` is needed
- what `new Dog("Gino")` does
- why `dog.bark()` works
- what `extends Dog` means
- why `super(name)` is needed
- what `this.job = job` does
- why `serviceDog` can use both `bark()` and `work()`

Part S: Sprint 20 Practice Tasks

55. Practice Task 1: Basic Class

Create a `Student` class with:

- `name`
- `score`
- a `passed()` method
- a `describe()` method

Starter:

```

class Student {
  constructor(name, score) {
    this.name = name;
    this.score = score;
  }

  passed() {
    return this.score >= 70;
  }

  describe() {
    return `${this.name} has a score of ${this.score}.`;
  }
}

const student = new Student("Sangeeth", 90);
console.log(student.describe());
console.log(student.passed());

```

Expected output:

```

Sangeeth has a score of 90.
true

```

56. Practice Task 2: Inheritance

Create an `OnlineStudent` class that extends `Student`.

It should add:

- `platform`
- an overridden `describe()` method

Starter:

```

class OnlineStudent extends Student {
  constructor(name, score, platform) {
    super(name, score);
    this.platform = platform;
  }

  describe() {

```

```
        return `${super.describe()} Platform: ${this.platform}.`;
    }
}

const onlineStudent = new OnlineStudent("Ada", 95, "Zoom");
console.log(onlineStudent.describe());
```

Expected output:

```
Ada has a score of 95. Platform: Zoom.
```

57. Practice Task 3: Static Method

Add a static method to `Student` :

```
static isValidScore(score) {
    return score >= 0 && score <= 100;
}
```

Use it like this:

```
console.log(Student.isValidScore(85));
console.log(Student.isValidScore(120));
```

Expected output:

```
true
false
```

58. Practice Task 4: Static Property

Add a static property:

```
static passingScore = 70;
```

Then use it inside `passed()` :

```
passed() {  
    return this.score >= Student.passingScore;  
}
```

This makes the passing score easy to change in one place.

Part T: Checkpoint Questions

59. Concept Questions

Answer these without looking at the notes:

1. What is a class?
 2. What is an instance?
 3. What does the `constructor` do?
 4. What does `new` do?
 5. What does `this` usually mean inside a class method?
 6. What is a method?
 7. What is an instance property?
 8. What is inheritance?
 9. What does `extends` do?
 10. What does `super()` do?
 11. Why must `super()` come before `this` in a child constructor?
 12. What is method overriding?
 13. What does `super.methodName()` do?
 14. What is a static method?
 15. What is a static property?
 16. What is the difference between an instance method and a static method?
 17. What is the difference between an instance property and a static property?
 18. What is a prototype, at a beginner level?
 19. Why should beginners avoid complex prototype manipulation?
 20. When should you use a class instead of a plain object?
-

Part U: Sprint 20 Summary

60. Final Summary

A class is a reusable blueprint for objects.


```
class Student {  
  constructor(name, score) {  
    this.name = name;  
    this.score = score;  
  }  
}
```

The `constructor` sets up each new object.

```
const student = new Student("Sangeeth", 90);
```

The `new` keyword creates an instance.

```
student
```

is an instance of `Student`.

`this` refers to the current object.

```
this.name
```

means the `name` property of this specific object.

Methods define behavior.

```
passed() {  
  return this.score >= 70;  
}
```

Inheritance lets one class reuse another class.

```
class OnlineStudent extends Student {}
```

`super()` calls the parent constructor.

```
super(name, score);
```

Static members belong to the class itself.

```
Student.isValidScore(90);  
Student.passingScore;
```

Prototype awareness means knowing that class methods are shared behind the scenes through JavaScript's prototype system.

Practical beginner rule:

Use classes when you need many similar objects with shared data shape and shared behavior.

Part V: One-Page Cheat Sheet

61. Sprint 20 Cheat Sheet

Basic class

```
class Student {  
  constructor(name) {  
    this.name = name;  
  }  
}
```

Create an instance

```
const student = new Student("Sangeeth");
```

Add a method

```
class Student {  
  constructor(name, score) {  
    this.name = name;  
    this.score = score;  
  }  
  
  passed() {  
    return this.score >= 70;  
  }  
}
```

Inheritance

```
class OnlineStudent extends Student {  
  constructor(name, score, platform) {  
    super(name, score);  
    this.platform = platform;  
  }  
}
```

Static method

```
class Student {  
  static isValidScore(score) {  
    return score >= 0 && score <= 100;  
  }  
}  
  
Student.isValidScore(90);
```

Static property

```
class Student {  
  static passingScore = 70;  
}  
  
console.log(Student.passingScore);
```

Main rules

Class = blueprint.
Instance = object made from class.
constructor = setup method.
new = creates instance.
this = current object.
extends = inherit from parent class.
super() = call parent constructor.
static = belongs to class, not instance.
prototype = behind-the-scenes shared behavior system.